scrambled-bytes

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

I sent my secret flag over the wires, but the bytes got all mixed up!

🧰 Tools Used

- `file` - Determine file type
- `strings` - Extract printable character sequences
- `exiftool` - Read metadata
- **Wireshark** - Visual packet analysis
- **tshark** - Command-line packet analysis
- **Scapy** - Python packet manipulation library
- **Python** - Custom reconstruction script

🔍 Complete Solution

Step 1: Download and Prepare

Download the files from the challenge page:

- `capture.pcapng` - Network packet capture
- `send.py` - Python script used to send the scrambled data

Step 2: Basic File Inspection

First, verify the file types:

```
file capture.pcapng send.py
```

Expected output:

```
capture.pcapng: pcapng capture file - version 1.0
send.py:        Python script, ASCII text executable
```

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings capture.pcapng | grep -i "picoCTF"
```

Output: (no flag, only references to picoctf.org)

The flag isn't directly visible.

Step 4: Inspect Metadata

```
exiftool capture.pcapng
```

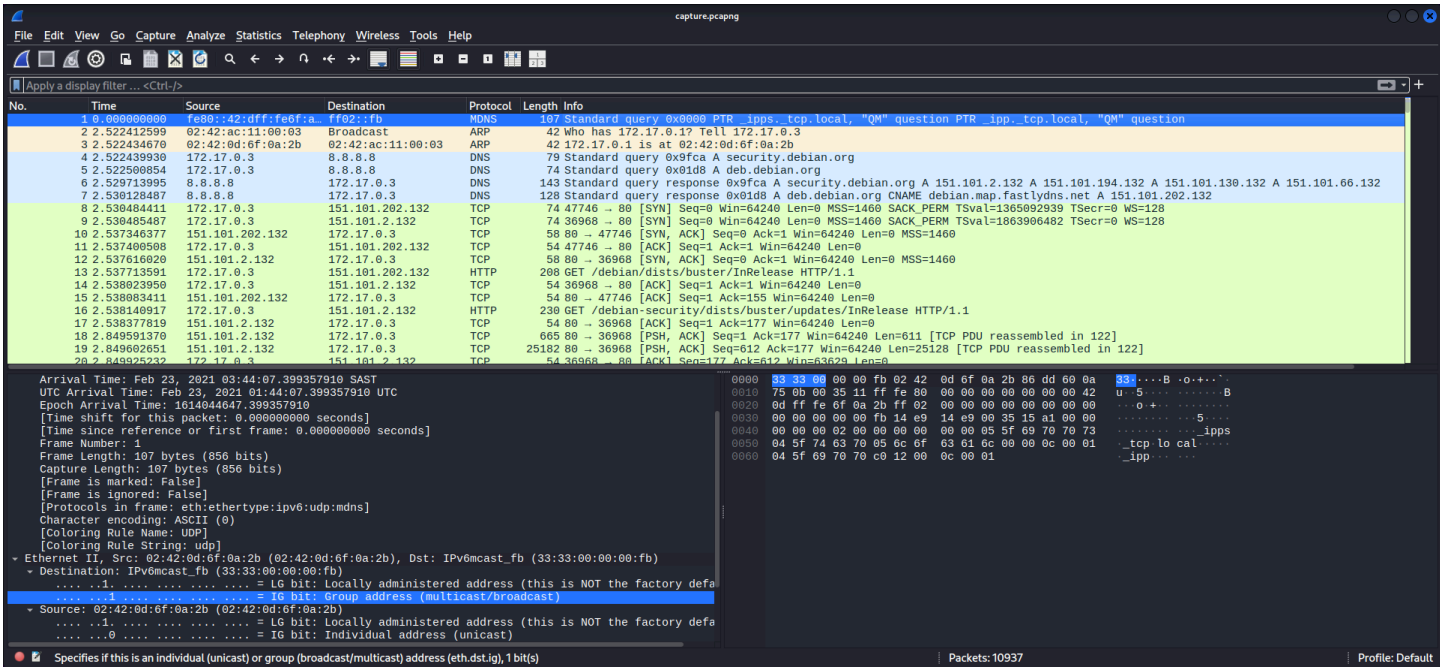
Output:

```
ExifTool Version Number         : 13.50
File Name                       : capture.pcapng
Directory                       : .
File Size                       : 30 MB
...
File Type                       : PCAPNG
User Application                : Dumpcap (Wireshark) 2.6.10
Link Type                       : IEEE 802.3 Ethernet
Operating System                : Linux 5.4.0-65-generic
Time Stamp                      : 2021:02:23 03:44:06.797738075+02:00
```

Nothing suspicious.

Step 5: Open with Wireshark

```
wireshark capture.pcapng
```



The capture contains UDP traffic. We need to understand how the data was sent.

Step 6: Analyze the Provided send.py Script

```
cat send.py
```

Output:

```python
#!/usr/bin/env python3

import argparse
from progress.bar import IncrementalBar

from scapy.all import *
import ipaddress

import random
```

```
from time import time

def check_ip(ip):
  try:
    return ipaddress.ip_address(ip)
  except:
    raise argparse.ArgumentTypeError(f'{ip} is an invalid address')

def check_port(port):
  try:
    port = int(port)
    if port < 1 or port > 65535:
      raise ValueError
    return port
  except:
    raise argparse.ArgumentTypeError(f'{port} is an invalid port')

def main(args):
  with open(args.input, 'rb') as f:
    payload = bytearray(f.read())
  random.seed(int(time()))
  random.shuffle(payload)
  with IncrementalBar('Sending', max=len(payload)) as bar:
    for b in payload:
      send(
        IP(dst=str(args.destination)) /
        UDP(sport=random.randrange(65536), dport=args.port) /
        Raw(load=bytes([b^random.randrange(256)])),
      verbose=False)
      bar.next()

if __name__=='__main__':
  parser = argparse.ArgumentParser()
  parser.add_argument('destination', help='destination IP address', type=check_ip)
  parser.add_argument('port', help='destination port number', type=check_port)
  parser.add_argument('input', help='input file')
  main(parser.parse_args())
```

Key observations:

Line	Action
`random.seed(int(time()))`	RNG seeded with the current time (seconds since epoch)

Line	Action
`random.shuffle(payload)`	Bytes are shuffled randomly
`sport=random.randrange(65536)`	Random source port (consumes one random number)
`b^random.randrange(256)`	Each byte is XORed with a random byte (consumes another random number)

The data is **scrambled in two ways**:

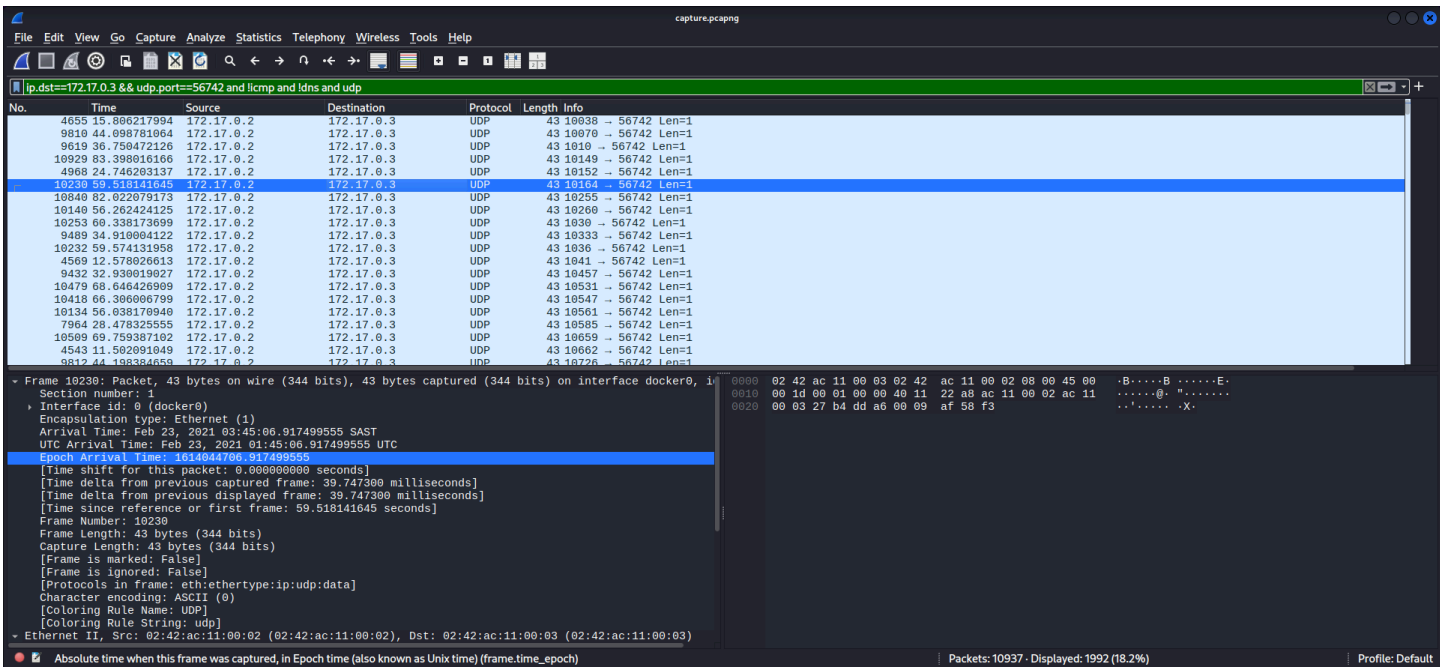
1. **Shuffling** - The byte order is randomized
2. **XOR encryption** - Each byte is XORed with a random value

Both operations depend on the **same RNG seed** (the timestamp when the script was run).

Step 7: Identify the Relevant Packets in Wireshark

Filter for UDP packets to the destination port (identified from the capture):

```
ip.dst==172.17.0.3 && udp.port==56742 && !icmp && !dns && udp
```



Each packet contains one byte of scrambled data in its payload.

Step 8: Extract the Seed

The seed is the **timestamp of the first packet** (when the script started). We can extract it from the PCAP.

Using `tshark`:

```
# Get the timestamp of the first UDP packet to port 56742
tshark -r capture.pcapng -Y "udp.dstport==56742" -T fields -e frame.time_epoch | head
-1
```

Output:

```
1614044650.913789387
```

The integer part (`1614044650`) is the seed.

Step 9: Extract Payload Data with tshark

```
# Extract all payload data (hex) from UDP packets
tshark -r capture.pcapng -Y "udp.dstport==56742" -T fields -e data | tr -d '\n' >
payload.hex

# Convert hex to raw bytes
xxd -r -p payload.hex > payload.raw
```

Or in one line:

```
tshark -r capture.pcapng -Y "udp.dstport==56742" -T fields -e data | xxd -r -p >
payload.raw
```

Step 10: Python Reconstruction Script

python

```
#!/usr/bin/env python3
from scapy.all import *
import random
```

```python
def reconstruct_flag(pcap_file, dst_port=56742):
    # Read packets from the pcap
    packets = rdpcap(pcap_file)

    # Filter UDP packets to the destination port
    udp_packets = [pkt for pkt in packets if pkt.haslayer(UDP) and pkt[UDP].dport ==
dst_port]

    if not udp_packets:
        print("No matching packets found!")
        return

    # Extract the seed from the first packet's timestamp
    seed = int(udp_packets[0].time)
    print(f"[*] Extracted seed: {seed}")

    # Extract XORed and shuffled payload
    data_bytes = []
    for pkt in udp_packets:
        if pkt.haslayer(Raw):
            data_bytes.append(pkt[Raw].load[0])  # Each packet contains 1 byte

    data = bytearray(data_bytes)
    print(f"[*] Extracted {len(data)} bytes")

    # Recreate the random sequence
    random.seed(seed)

    # 1. Recreate the shuffle order
    order = list(range(len(data)))
    random.shuffle(order)

    # 2. Undo XOR (consume port random numbers first, then XOR values)
    for i in range(len(data)):
        random.randrange(65536)  # Consume the random source port
        xor_val = random.randrange(256)
        data[i] ^= xor_val

    # 3. Inverse shuffle
    original_bytes = bytearray(len(data))
    for original_index, shuffled_index in enumerate(order):
        original_bytes[shuffled_index] = data[original_index]

    return original_bytes

if __name__ == "__main__":
```

```
result = reconstruct_flag("capture.pcapng", dst_port=56742)

# Save as PNG
with open("flag.png", "wb") as f:
    f.write(result)
print("[*] Saved flag.png")

# Also try to decode as text
try:
    print(result.decode('utf-8', errors='ignore'))
except:
    pass
```

Output:

```
[*] Extracted seed: 1614037446
[*] Extracted 562 bytes
[*] Saved flag.png
```

Step 11: View the Reconstructed Image

```
xdg-open flag.png   # Linux
open flag.png       # macOS
start flag.png      # Windows
```

The flag is visible in the image!

Step 12: Alternative - Using tshark Only (No Scapy)

If you don't want to use Scapy, extract data with `tshark` and process with a simpler Python script:

```
# Extract timestamps and payloads
tshark -r capture.pcapng -Y "udp.dstport==56742" -T fields -e frame.time_epoch -e data
```

Output (example):

```
1614037446.797738075   57
1614037446.798123456   a3
...
```

Now use a Python script that reads this output:

python

```
#!/usr/bin/env python3
import random
import sys

# Parse input: each line is "timestamp hex_byte"
lines = [line.strip().split() for line in sys.stdin]
timestamps = [float(line[0]) for line in lines]
data_hex = [line[1] for line in lines]

seed = int(timestamps[0])
data = bytearray([int(h, 16) for h in data_hex])

random.seed(seed)

order = list(range(len(data)))
random.shuffle(order)

for i in range(len(data)):
    random.randrange(65536)
    xor_val = random.randrange(256)
    data[i] ^= xor_val

original = bytearray(len(data))
for orig_idx, shuffled_idx in enumerate(order):
    original[shuffled_idx] = data[orig_idx]

with open("flag.png", "wb") as f:
    f.write(original)
```

```
print("Saved flag.png")
```

Run it:

```
tshark -r capture.pcapng -Y "udp.dstport==56742" -T fields -e frame.time_epoch -e data
| python3 reconstruct.py
```

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, the reconstructed PNG will display the complete flag.

🧠 Key Concepts

Concept	Description
Random Number Generator (RNG) Seeding	The Python `random` module generates pseudo-random numbers based on a seed. The same seed produces the same sequence.
Shuffling	`random.shuffle()` reorders the bytes of the payload.
XOR Encryption	Each byte is XORed with a random byte to obscure the data.
UDP Payload	Each UDP packet carries exactly one byte of scrambled data.
Timestamp as Seed	The script used `int(time())` as the seed, which we can recover from the first packet's timestamp.
Reverse Engineering	By understanding the scrambling process, we can reverse it step by step.

💡 Pro Tips

1. **The seed is the integer part of the first packet's timestamp** - This is critical for regenerating the same random sequence.
2. **Random number consumption order matters** - The script consumes:
   - One random for the source port (not used in decoding)
   - One random for XOR (used to decode)

   You must consume them in the same order.
3. **Scapy is powerful for PCAP analysis** - It gives you direct access to packet timestamps and payloads.
4. **tshark can extract data without Scapy** - Use `-T fields -e data` to extract payloads.
5. **The final output is a PNG** - The reconstructed bytes form a valid PNG image.
6. **Verify the file magic bytes** - After reconstruction, the file should start with `89 50 4E 47` (PNG signature).

🔗 Alternative Tools

Tool	Purpose
Scapy	Python library for packet manipulation and analysis
tshark	Command-line Wireshark for field extraction
xxd	Hex dump and conversion utility

🏁 Conclusion

The `send.py` script scrambled a file (the flag image) by:

1. Seeding the RNG with the current timestamp
2. Shuffling the bytes

3. XORing each byte with a random value
4. Sending each byte as a separate UDP packet

To recover the flag, we:

1. Extracted the first packet's timestamp to get the RNG seed
2. Extracted all UDP payloads in order
3. Recreated the same random sequence
4. Undid the XOR operation
5. Reversed the shuffle
6. Saved the reconstructed bytes as a PNG image

This challenge demonstrates:

- **RNG-based scrambling** - Using pseudo-random sequences for obfuscation
- **Packet analysis** - Extracting data from UDP packets
- **Reverse engineering** - Understanding and reversing a custom scrambling algorithm

Key takeaway: When data is scrambled with a pseudo-random sequence, the security depends entirely on the secrecy of the seed. If you can recover the seed (from timestamps or other metadata), you can reverse the process and recover the original data.